



[Products](#) / [DataX](#) / [DataX Manual](#) / [Configurations](#)

Configurations

Configurations

Available Configurations

Navigation

Data Connectors

Purpose

Common Types / Categories

What to Configure

Typical Lifecycle

Common Operations

Reference

Creating a Data Connector

Connection Details & Authentication (Based on Connector Category)

Edit or Delete a Data Connector

Lookup Sources

Purpose

Typical Use Cases

Key Configuration Fields

Creating a Lookup Source

Edit or Delete a Lookup Source

Data Formats

Purpose

Typical Capabilities

What you configure in a Data Format

Supported components (high-level)

2. Data Format

Key Features

2.1 Create a New Data Format

Data Details Configuration

Job Details

Reader Details

Transformation Details

Writer Details

2.3 Edit & Delete Data Format

Data Sets

Purpose

Key Configuration

Common Guardrail

3.1 Create Data Set

3.2 Edit & Delete Data Set

DataX configuration is typically performed by a **Client Admin**.

Available Configurations

DataX setup now typically spans the following configuration areas:

- **Data Connectors** (🔌) - connection/authentication to external systems
- **Data Formats** (📄) - parsing/mapping/validation rules for incoming data
- **Data Sets** (🗃️) - the logical container that links one or more Data Formats
- **Lookup Sources** (🔍) - master and reference data tables used to enrich records during transformation
- **Integration Systems** - external business or source systems
- **PII Types** - reusable sensitive-data categories and their matching patterns
- **Data Discovery** - guided scanning and format generation for database sources

For screen-by-screen guidance on the newer database-onboarding flow, see:

- [Integration Systems](#)
- [PII Types](#)
- [Data Discovery](#)

Navigation

- Admin > DataX > Data Connectors
- Admin > DataX > Integration Systems
- Admin > DataX > PII Types
- Admin > DataX > Data Discovery
- Admin > DataX > Data Formats
- Admin > DataX > Data Sets
- Admin > DataX > Lookup Sources

Data Connectors 🔌

Purpose

A Data Connector stores connection and authentication details for external systems.

Common Types / Categories

- **Connector Type:** Source, Target, or Source and Target
- **Category:** HTTP, Database, Email, Object Storage, SFTP

What to Configure

Area	What it Controls
Connection Details	Host/port/database, base URL + headers, email host/port, SFTP directory, bucket/prefix/region, etc.
Authentication Details	Basic, Bearer, OAuth, Certificate, AWS Credentials
Fetch Details (if applicable)	Cron expression and scheduling metadata

Typical Lifecycle

1. Create a connector for your source/target system
2. Validate connection + authentication details
3. For **Source** connectors, configure the fetch schedule (cron) when applicable
4. Reuse the connector in one or more Data Formats

Common Operations

- **Create:** Admin selects connector category and type, then enters connection + authentication settings
- **Edit:** Admin updates editable fields and saves
- **Delete:** Admin removes a connector when it is no longer referenced by formats/sets

Reference

The **Data Connector** serves as the integration layer with external systems. It supports integration with various protocols and services such as **Databases**, **HTTP APIs**, **Email**, **Object Storage**, and **SFTP**, and can be used as a **Source**, **Target**, or **both**.

Creating a Data Connector

1. Navigate to **DataX > Admin**, and click on the **DataConnectors** tile.
2. Click on **Create DataConnector**.

3. Enter the following details:

- **Name:** A unique name for the connector.
- **Description:** Optional, but useful for context.

4. Select the **Category** from the dropdown:

- Options include: Database , HTTP , Email , Object Storage , SFTP , etc.

5. Choose the **Connector Type**:

- Options: Source , Target , or Source and Target

6. Enter the **Connection Details** and **Authentication Credentials** These fields vary based on the selected **Category**.

See examples below:

Category	Connection & Authentication Fields
Database	* Host * Port * Database Name * Username * Password
HTTP	* Endpoint URL * HTTP Method * Headers * Authentication Type (Basic / API Key / OAuth)
Email	* SMTP/IMAP Configuration * Email ID * Password
SFTP	* Host * Port * Username * Private Key or Password

NOTE

Once configured, the Data Connector becomes available for use in **Jobs** or **Pipelines** that require integration with the respective source or target systems.

Connection Details & Authentication (Based on Connector Category)

Each Data Connector category comes with specific configuration fields depending on the selected Connector Type — **Source**, **Target**, or **both**.

NOTE

For Source connectors, a **Cron Expression** is required to schedule data fetch intervals.

HTTP Connector

Used for integrating with REST APIs or web services.

- **Cron Expression:** Required only for Source type.
- **Connection Details:**
 - **Base URL:** The endpoint of the API.

- **Header Name / Value:** Optional key-value pairs to include in the request header (e.g., API Key, Content-Type).
- **Authentication:**
 - **Authentication Name:** A user-defined name for the credentials.
 - **Authentication Description:** Description for clarity.
 - **Authentication Type:** Basic, Bearer Token, OAuth, Certificate, or AWS Credentials.

Connection Mode for HTTP Data Formats

When an HTTP connector is used as a **source** in DataX, the linked Data Format controls whether the request executes from the backend runtime or from the browser.

Connection Mode	Behavior
REMOTE	Default mode. DataX backend executes the configured HTTP request during ingestion.
LOCAL	Manual ingestion flow where the browser executes the HTTP call first, then submits the endpoint response to DataX as trigger payload.

NOTE

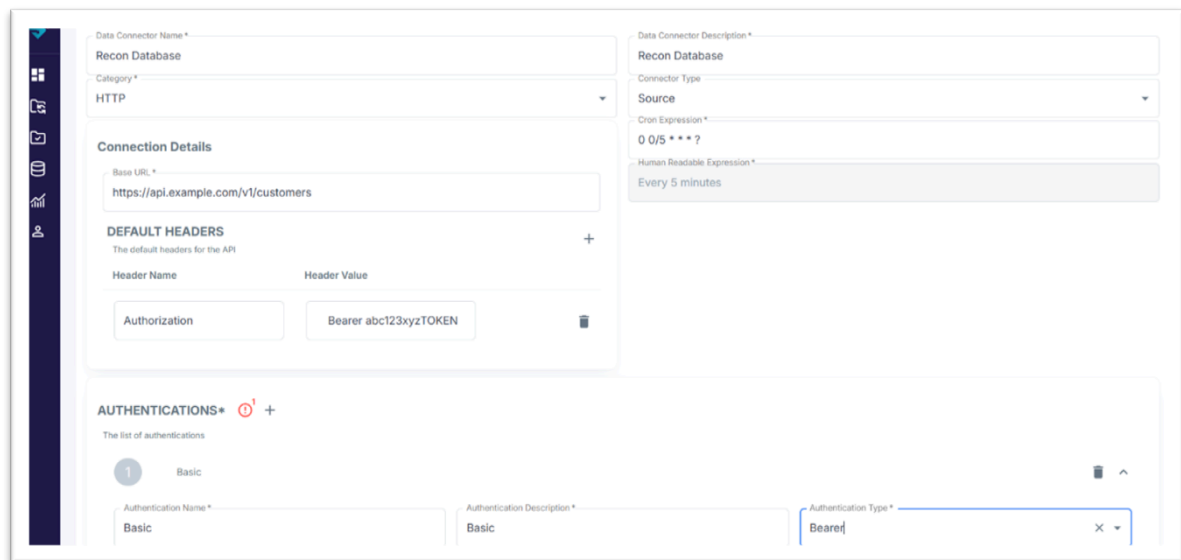
LOCAL mode is intentionally constrained:

- Only `POST` requests are supported.
- Authentication must be `ANONYMOUS`.
- The endpoint response must be a flat JSON array.

Request Body Behavior in Manual Ingestion

The UI input controls are derived from `HttpReaderConfig.requestBodyType`:

Request Body Type	Manual Ingestion Behavior
FILE	User uploads a file in Manual Ingestion. For <code>REMOTE</code> , DataX uploads and forwards it. For <code>LOCAL</code> , browser sends the file directly to the endpoint and submits the response back to DataX.
JSON	Request body is pre-configured in the Data Format. Manual trigger does not prompt for a body.
FORM_DATA	Form body is pre-configured in the Data Format. Manual trigger does not prompt for per-run values.
NONE	No request body is sent.



Data Connector Name *
Recon Database

Category *
HTTP

Connection Details

Base URL *
https://api.example.com/v1/customers

DEFAULT HEADERS
The default headers for the API

Header Name	Header Value
Authorization	Bearer abc123xyzTOKEN

AUTHENTIFICATIONS* 1 +
The list of authentications

Authentication Name *	Authentication Description *	Authentication Type *
Basic	Basic	Bearer

Data Connector Description *
Recon Database

Connector Type
Source

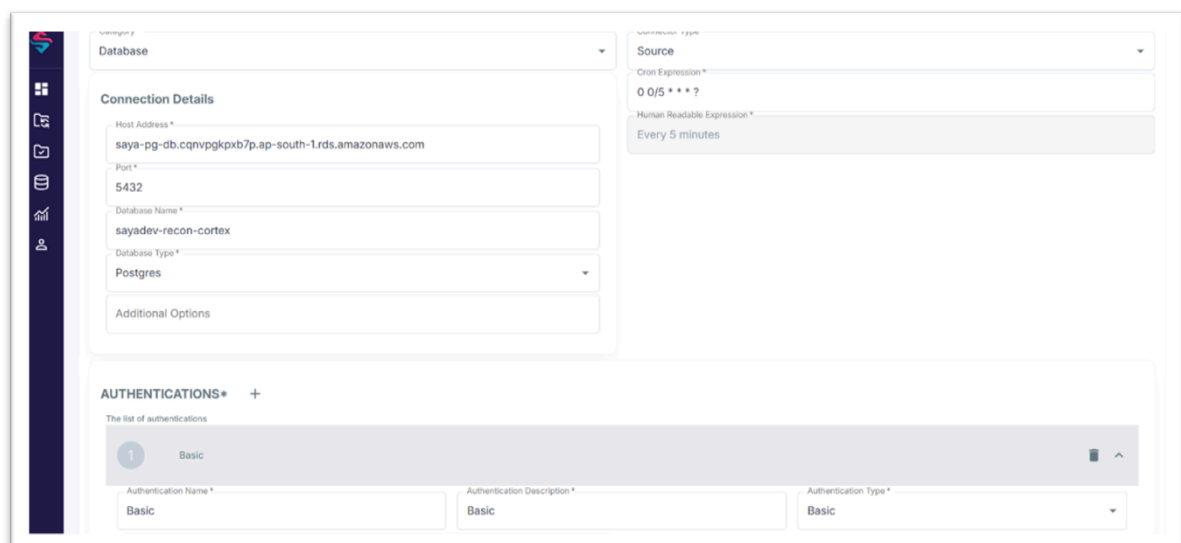
Cron Expression *
0 0/5 * * * ?

Human Readable Expression *
Every 5 minutes

Database Connector

Supports connection with relational databases.

- **Cron Expression:** Required only for Source type.
- **Connection Details:**
 - **Host Address:** IP or domain of the DB server.
 - **Port:** e.g., 5432 for PostgreSQL.
 - **Database Type:** Supported types like PostgreSQL, Oracle, MSSQL, etc.
- **Authentication:**
 - **Authentication Name:** Reference label.
 - **Authentication Description:** Context of credentials.
 - **Authentication Type:** Basic, OAuth, Certificate, or DB-specific credential methods.



Database

Connection Details

Host Address *
say-pg-db.cqnvpgkpxb7p.ap-south-1.rds.amazonaws.com

Port *
5432

Database Name *
sayadev-recon-cortex

Database Type *
Postgres

Additional Options

AUTHENTIFICATIONS* 1 +
The list of authentications

Authentication Name *	Authentication Description *	Authentication Type *
Basic	Basic	Basic

Data Connector Description *
Source

Connector Type
Source

Cron Expression *
0 0/5 * * * ?

Human Readable Expression *
Every 5 minutes

Email Connector

Used to read data from email inboxes.

- **Cron Expression:** Required only for `Source` type.
- **Connection Details:**
 - **Host Address:** IMAP/SMTP server (e.g., `imap.gmail.com`).
 - **Port:** Port number (e.g., 993 for IMAP).
 - **Email Address:** Email account to connect.
 - **Email Labels:** Optional filters like `inbox`, `sent`, etc.
- **Authentication:**
 - **Authentication Name:** Custom identifier.
 - **Authentication Description:** Optional note.
 - **Authentication Type:** Basic, OAuth, or App-specific Passwords.

Object Storage Connector

Used to connect with cloud storage systems like AWS S3, GCS, etc.

- **Cron Expression:** Required only for `Source` type.
- **Connection Details:**
 - **Bucket Name:** Target bucket.
 - **Prefix:** Directory or file path prefix.
 - **Region:** Cloud region where the storage bucket resides.
- **Authentication:**
 - **Authentication Name:** User-defined identifier.
 - **Authentication Description:** Clarifies purpose or usage.
 - **Authentication Type:** AWS Credentials, Bearer Token, Basic, Certificate.

The screenshot shows the configuration page for an AWS S3 Data Connector. The interface is divided into several sections:

- Data Connector Name:** AWS S3
- Category:** Object Storage
- Data Connector Description:** AWS S3
- Connector Type:** Source and Target
- Connection Details:**
 - Bucket Name:** saya-file-ingestion
 - Prefix:** Saya
 - Region:** ap-southeast-1
- AUTHENTIFICATIONS:**
 - 1 SAYA S3 Storage** (indicated by a red circle with a '1')
 - Authentication Name:** SAYA S3 Storage
 - Authentication Description:** SAYA S3 Storage
 - Authentication Type:** AWS Credentials

SFTP Connector

Used for secure file transfer between systems.

- **Cron Expression:** Required only for Source type.
- **Connection Details:**
 - **Host Address:** SFTP server address.
 - **Port:** Typically 22.
 - **Root Directory:** Base directory for file operations.
- **Authentication:**
 - **Authentication Name:** Unique name.
 - **Authentication Description:** Purpose or environment context.
 - **Authentication Type:** Basic (username/password), Certificate, etc.

The screenshot shows the configuration page for an SFTP Data Connector. The interface is divided into several sections:

- Data Connector Name:** Recon Database
- Category:** SFTP
- Data Connector Description:** Recon Database
- Connector Type:** Source
- Connection Details:**
 - Host Address:** sftp.example.com
 - Port:** 22
 - Root Directory:** /data/incoming/reports/
- Authentication:**
 - Cron Expression:** 0 0/5 * * *
 - Human Readable Expression:** Every 5 minutes
- AUTHENTIFICATIONS:**
 - 1 Basic** (indicated by a red circle with a '1')
 - Authentication Name:** Basic
 - Authentication Description:** Basic
 - Authentication Type:** Certificate

Edit or Delete a Data Connector

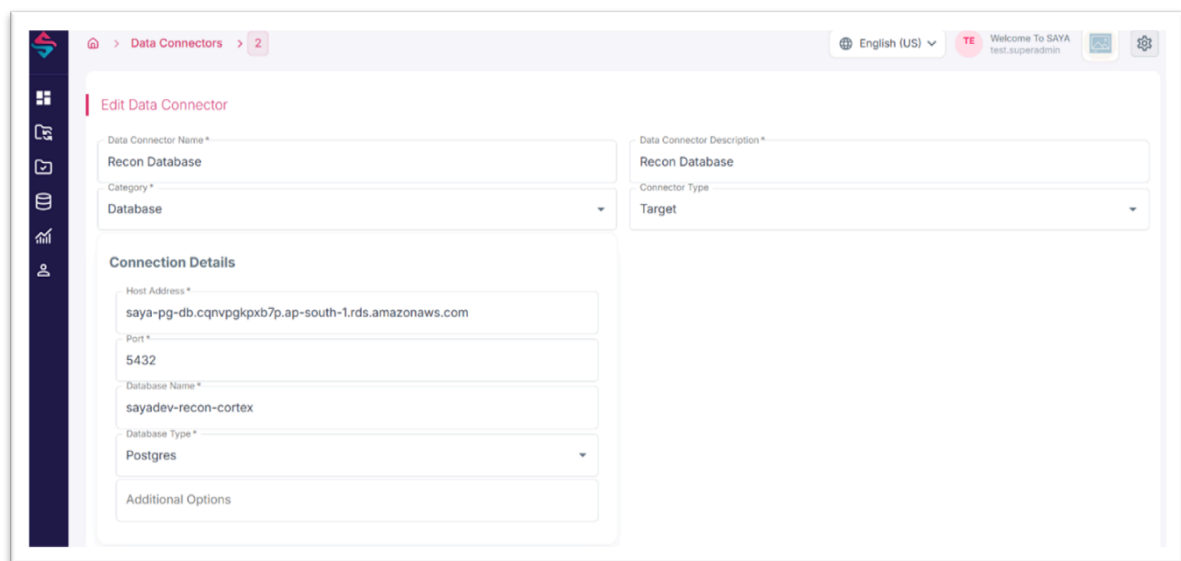
You can modify or remove an existing Data Connector from the DataX admin panel.

Steps to Edit a Data Connector

1. Navigate to DataX > Admin , and click the **Data Connectors** tile.
2. Click on the Data Connector you want to edit.
3. Update editable fields as required.
4. Click **Submit**.

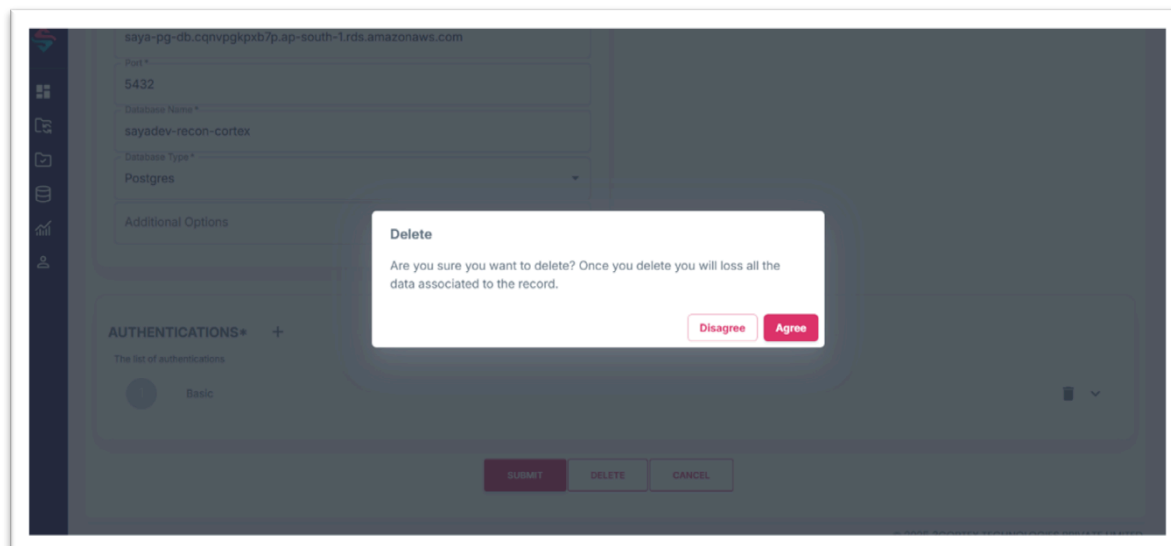
NOTE

Only fields allowed by the system can be modified. The system will automatically save and apply your changes.



Steps to Delete a Data Connector

1. Navigate to the **Data Connectors** tile under DataX .
2. Select the Data Connector to delete.
3. Click the **Delete** button.
4. A confirmation dialog will appear.
5. Click **Agree** to delete or **Disagree** to cancel.



Lookup Sources Q

Purpose

A Lookup Source defines a master or reference data table that DataX pipelines can query during transformation to enrich output records with resolved values. Unlike Data Connectors (which describe how data arrives at the pipeline), a Lookup Source defines **what** reference data is available and **how** it is loaded into memory before processing begins.

Lookup Sources are reusable. A single entry — such as `Bond Master` or `Holiday Calendar` — can be referenced by multiple Data Formats and by multiple Lookup Transformation fields within the same format.

Typical Use Cases

- Resolving a bond classification (bond type, issuer) from an ISIN against a `Bond Master` table
- Deriving a settlement date by checking whether a date is a public holiday in a given country
- Enriching transactions with the reference FX rate for a currency pair on a given value date
- Fetching issuer credit ratings from an `Issuer Master` by issuer code

Key Configuration Fields

Field	Required	Description
Short Name	Yes	A brief human-readable name displayed in dropdowns and pipeline configuration screens (e.g. <code>Bond Master</code> , <code>Holiday Calendar</code>).

Field	Required	Description
Long Name	Yes	A descriptive full name providing context (e.g. Active Bond Master – ISIN to Classification and Issuer Mapping).
Data Connector	Yes	The database Data Connector that supplies the connection and credentials for the reference data source. Must be a database-type connector.
Load Strategy	Yes	Controls how the data is loaded into memory. <code>PRELOAD_FULL</code> fetches the entire result set once at pipeline start and holds it in memory for the duration of the step. This is the only supported strategy.
Query	Yes	A <code>SELECT</code> statement that retrieves the complete reference data set. Only <code>SELECT</code> statements are accepted — any query containing <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>DROP</code> , <code>ALTER</code> , <code>EXEC</code> , or <code>CALL</code> is rejected at save time.
Max Rows	No	An upper bound on the number of rows the query may return. If the result exceeds this value, the pipeline fails before processing any records. When not set, a platform-level limit of 500,000 rows applies.

Creating a Lookup Source

1. Navigate to `DataX > Admin`, and click on the **Lookup Sources** tile.
2. Click on **Create Lookup Source**.
3. Enter a **Short Name** and **Long Name**.
4. Select a database **Data Connector** from the dropdown.
5. Set the **Load Strategy** — `PRELOAD_FULL` is the default and only available option.
6. Enter a valid `SELECT` **Query**. The result of this query forms the in-memory reference table available to pipeline transformation steps.
7. Optionally set a **Max Rows** limit.
8. Click **Submit**.



Create Lookup Source

Query Guidelines

- Only `SELECT` statements are accepted. Queries containing DML or DDL keywords are rejected when you save.
- Use date-scoped queries to keep the loaded data set within a practical size. For example, a Holiday Calendar query filtered to the current year loads far fewer rows than an unfiltered query

over all historical data.

Use Case	Example Query
Bond Master (all active bonds)	<code>SELECT ISIN, BOND_TYPE, ISSUER_CODE, CURRENCY FROM BOND_MASTER WHERE ACTIVE = 1</code>
Holiday Calendar (current year)	<code>SELECT COUNTRY_CODE, HOLIDAY_DATE, IS_HOLIDAY, HOLIDAY_NAME FROM HOLIDAY_CALENDAR WHERE CAL_YEAR = :CURRENT_YEAR</code>
FX Rate Master (rolling window)	<code>SELECT CCY_PAIR, RATE_DATE, RATE FROM FX_RATES WHERE RATE_DATE >= :WINDOW_START</code>

Edit or Delete a Lookup Source

To edit or delete a Lookup Source, go to the respective entry and click on the corresponding icon (edit or delete).

WARNING

Deleting a Lookup Source that is referenced by a Data Format's Lookup Transformation configuration will cause those pipeline steps to fail at startup. Remove all referencing Lookup Transformation entries from your Data Formats before deleting a source.



Edit Lookup Source

Data Formats

Purpose

A Data Format defines how incoming data is interpreted (parsing + mapping + validation).

Typical Capabilities

- Supported file extensions (e.g., CSV, JSON, XML)
- File-type handling (Plain Text, Encrypted, Compressed, Derive Automatically)
- Name-based identification using patterns
- Field mapping and conversions

What you configure in a Data Format

At minimum, a Data Format typically includes:

- Source connector and (optional) sink connector

- File type handling (including **Derive Automatically**)
- Name identifier + pattern matching (used during upload validation)
- Allowed file extensions

Supported components (high-level)

Available components can depend on connector category (for example, database ingestion enables JDBC readers).

Readers (examples):

- CSV Reader
- XLS/XLSX Reader
- Grok Reader
- JSON Reader
- SWIFT MX Reader
- SWIFT MT Reader
- JDBC Reader (for database ingestion)

Processors / Transformers (examples):

- Add Field Alias
- Format Amount
- Derive Amount Type
- Format Date/Time
- Add Default Value
- Map to New Values
- Remove Quote(s)
- Replace Characters / Remove Characters
- Format Scientific Numbers
- Add Recon Metadata
- Compute Record Checksum
- Compose New Fields
- Scale Decimal Numbers
- Split Fields
- Mask Field Values
- Normalize XML

- Construct XML

Writers (examples):

- CSV Writer
- HTTP Writer
- JDBC Writer

NOTE

Data Formats are linked to Data Sets; a Data Set can have one or more formats.

2. Data Format

The Data Format module allows users to configure the structure and transformation of incoming data, ensuring it is compatible with processing pipelines and outputs. This includes setting up ingestion sources, transformation rules, and output configurations.

Key Features

- Data Ingestion: Supports importing data from various sources, including CSV, PDF, JSON, and more.
- Transformation Configuration: Enables operations such as:
 - Merging columns
 - Adding new columns
 - Modifying data formats
- Data Output: Allows configuration of output targets such as:
 - JDBC (database connections)
 - CSV

2.1 Create a New Data Format

Steps to Create:

1. Click on "Create New Data Format".
2. Provide a unique name and an optional description.
3. Choose the source connector and sink connector from the dropdown menus.
4. Select the file type and define a name identifier (this helps identify files with a matching name pattern during upload).
5. Click "Submit" to save the new data format.

Data Details Configuration

Field	Description
File Type	Choose from: • Plain Text - For standard readable files like CSV, TXT, JSON, etc. • Encrypted - For secure files that need decryption • Compressed - For zipped or archived files like .zip or .gz. • Derive Automatically - System auto-detects the file type based on metadata or extension.
Name Identifier	A unique string that should appear in the filename. This is used for validation during dataset uploads in the DataX Workspace.
Pattern Type	Select the pattern matching logic. Currently supported: Grok
Allowed Extensions	Add multiple file extensions. Supported types include: • .csv • .json • .pdf • .txt • .xls, .xlsx • .xml
Name Pattern	This will be generated automatically based on the provided name identifier and selected extensions.

The screenshot shows the 'Data Details' configuration page. At the top, there are dropdowns for 'Source Connector' (AWS S3) and 'Sink Connector' (Recon Database). Below these are tabs for 'Data Details', 'Job Details', 'Reader Details', 'Transformation Details', and 'Writer Details'. The 'Data Details' tab is active. It contains a 'File Type' dropdown set to 'Plain Text'. Under 'Name Details', there is a 'Name Identifier' field with 'BNK' and a 'Pattern Type' dropdown with 'Grok'. Below these are two sections: 'ALLOWED EXTENSIONS' with a list of supported extensions (JSON) and 'NAME PATTERNS' with a list of patterns to match the storage name. At the bottom, there are 'SUBMIT' and 'CANCEL' buttons.

Job Details

The Job Details section allows you to configure how a file will be processed in the system. It defines the job type, parallel processing settings, and how data should be chunked during ingestion.

- **Component Name:** Type of job to be executed (e.g., File Processing Job).
- **Grid Size:** Number of parallel threads or tasks for efficient processing.
- **Chunk Size:** Number of records to be processed per batch for optimized performance.

The screenshot shows the 'Job Details' configuration page. At the top, there are dropdowns for 'Source Connector' (AWS S3) and 'Sink Connector' (Recon Database). Below these are tabs for 'Data Details', 'Job Details', 'Reader Details', 'Transformation Details', and 'Writer Details'. The 'Job Details' tab is active. It contains a 'JOBS' section with a '+' icon and a list of job components. The first component is 'File Processing Job'. Below this, there is a 'Configuration' section with 'Grid Size' (2) and 'Chunk Size' (20000). At the bottom, there are 'SUBMIT' and 'CANCEL' buttons.

Reader Details

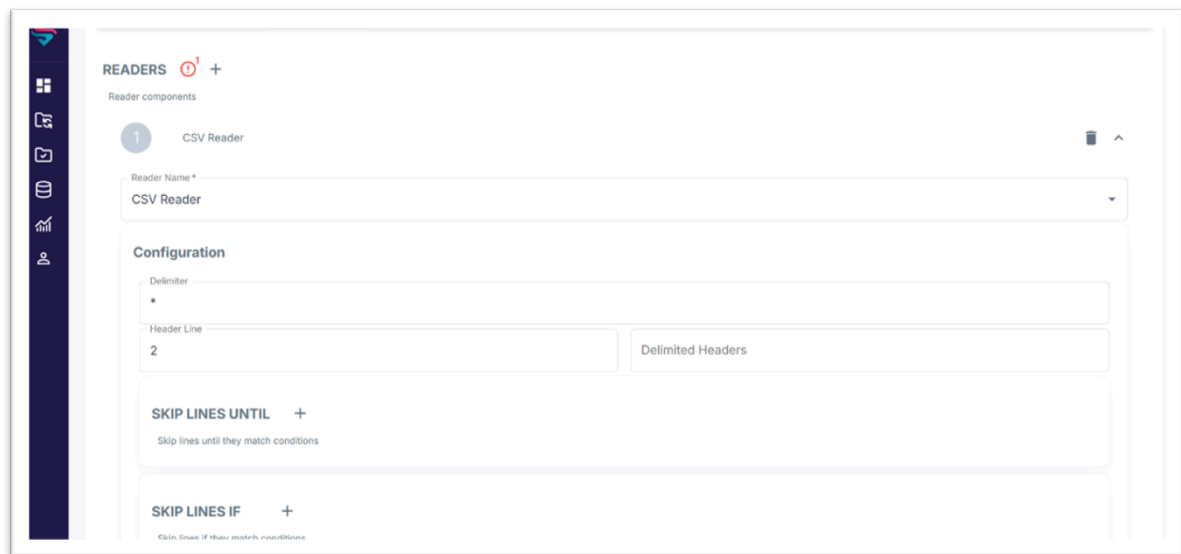
The Reader Details section defines how input files are parsed based on their type. Each file format (CSV, Excel, JSON, etc.) requires specific configurations to correctly interpret the data before transformation.

Reader Components and Their Configuration Options

CSV Reader

Configure how CSV files are read:

- **Delimiter:** Symbol used to separate values (e.g., comma, tab).
- **Header Line:** Line number where column headers are located.
- **Delimited Headers:** Indicates if headers themselves are delimited.
- **Skip Lines Until / If / After:** Set logic to skip lines based on:
 - Line Type: Header/Data/Any
 - Starts With / Contains: Keywords or patterns to match



The screenshot shows the 'READERS' configuration page in the SAYA Platform. A sidebar on the left contains icons for various components. The main area is titled 'READERS' with a plus icon and a 'Reader components' section. A 'CSV Reader' component is listed with a '1' icon. Below this, the 'Reader Name' is set to 'CSV Reader'. The 'Configuration' section includes a 'Delimiter' field with a '*' symbol, a 'Header Line' field with the value '2', and a 'Delimited Headers' checkbox. Below these are two sections for skipping lines: 'SKIP LINES UNTIL' and 'SKIP LINES IF', each with a plus icon and a description 'Skip lines until they match conditions'.

XLS/XLSX Reader

Read data from Excel files:

- **Sheet Names:** Specify the Excel sheet(s) to read from.
- **Header Lines:** Line number(s) containing column headers.
- **Skip Lines Until / If / After:** Same configuration as CSV reader.

Reader components

1 XLS/XLSX Reader

Reader Name *

XLS/XLSX Reader

Configuration

SHEET NAMES

The list of sheets for processing. Empty processes all sheets

Sheet1

+ Header Line

SKIP LINES UNTIL +

Skip lines until they match conditions

SKIP LINES IF +

Skip lines if they match conditions

Grok Reader

Used for custom log/text formats:

- **Header Line:** Indicates the header row.
- **Grok Pattern:** Pattern used to extract values from each line.

READERS +

Reader components

1 Grok Reader

Reader Name *

Grok Reader

The name of the reader

Configuration

Header Line *

Grok Pattern *

SUBMIT DELETE CANCEL

JSON Reader

Supports structured JSON files:

- **Flatten Nested Objects:** Converts nested fields into flat fields.
- **Flatten Nested Arrays:** Expands arrays into individual rows.

The screenshot shows the 'READERS' configuration page for a 'JSON Reader'. The 'Reader Name' is set to 'JSON Reader'. Under the 'Configuration' section, both 'Flatten Nested Objects' and 'Flatten Nested Arrays' are checked. At the bottom, there are 'SUBMIT', 'DELETE', and 'CANCEL' buttons.

Swift MX Reader

Parses SWIFT XML messages:

- **Message Type:** Select from CAMT 053 or CAMT 054.

The screenshot shows the 'READERS' configuration page for a 'SWIFT MX Reader'. The 'Reader Name' is set to 'SWIFT MX Reader'. Under the 'Configuration' section, the 'Message Type' dropdown is open, showing 'CAMT 053' and 'CAMT 054' as options. At the bottom, there are 'SUBMIT', 'DELETE', and 'CANCEL' buttons.

Swift MT Reader

Parses SWIFT plain-text messages:

- **Message Type:** Specify MT message format (e.g., MT940).
- **Include Fields:** Define which fields to extract.
- **Message Delimiter:** Separator used between messages.
- **Tag Name:** Tag identifiers for data extraction.

The screenshot shows the 'READERS' configuration page with a sub-header 'Reader components'. A single component, '1 SWIFT MT Reader', is listed. Below this, the 'Reader Name' is set to 'SWIFT MT Reader'. The 'Configuration' section includes a 'Message Type' dropdown menu with 'MT950' and 'MT940' as options. To the right of this dropdown is an 'INCLUDE FIELDS' section with the text 'The list of fields for inclusion' and a '+ Add' button. At the bottom of the configuration area are three buttons: 'SUBMIT', 'DELETE', and 'CANCEL'.

Transformation Details

Transformers allow you to clean, reshape, and enrich the data during file ingestion. Below are the commonly used transformers and their configuration options:

Add Field Alias

Rename fields to more user-friendly names.

- **Source Field:** The original field name in the file.
- **Alias Field:** The new name to assign.

Example: Rename `txn_id` to Transaction ID.

The screenshot shows the 'ROW TRANSFORMERS' configuration page with a sub-header 'Row-level transformer components'. A single component, '1 Add Field Alias', is listed. Below this, the 'Transformer Name' is set to 'Add Field Alias'. The 'Configuration' section includes a 'Source Field' input field with 'Amount' entered and an 'Alias Field' input field with 'Total Amount' entered. To the right of the 'Alias Field' input is a red '+ Add' button. At the top of the page, there are tabs for 'Data Details', 'Job Details', 'Reader Details', 'Transformation Details' (which is active), and 'Writer Details'.

Format Amount

Standardize numeric values by removing symbols or formatting.

- **Source Fields:** Fields containing amount values to clean.

Example: Convert "1,200.00" to 1200.00.

The screenshot shows the 'ROW TRANSFORMERS' configuration page with the 'Transformation Details' tab selected. A single transformer named 'Format Amount' is listed. The configuration section shows 'SOURCE FIELDS' with a list containing 'Amount'.

Derive Amount Type

Calculate a unified amount field based on credit/debit structure.

- **Is Debit Amount Signed:** Enable if debit values are negative.
- **Credit Amount Column:** Column name for credit amounts.
- **Debit Amount Column:** Column name for debit amounts.
- **Total Amount Column:** Column with signed totals (if any).
- **Derived Amount Column:** Field where the result will be stored.

Example: Combine debit = 100 and credit = 0 to derive amount = -100.

The screenshot shows the 'ROW TRANSFORMERS' configuration page with the 'Derive Amount Type' transformer selected. The configuration section includes:

- ☒ **Is Debit Amount Signed**
- Credit Amount Column:** Cr
- Debit Amount Column:** Dr
- Total Amount Column:** Ttl Amount (highlighted in blue)
- Derived Amount Column:** (empty)

 A note below the total amount column states: 'Column name containing total amount with sign'.

Format Date/Time

Convert date and time into desired formats and timezones.

- **Source Field:** Field containing the date/time.
- **Trim Size:** Trim input if needed (e.g., first 10 chars).
- **Input Format / Output Format:** Define custom formats (e.g., yyyy-MM-dd).
- **Time Input Format / Time Output Format:** Handle time-specific conversions.
- **Output TimeZone:** Adjust to desired timezone.

Example: Convert 2024-01-01T13:00:00Z to 01-Jan-2024 06:30 PM IST.

image::Time Transformations, width=600]

Add Default Value

Set a default when a field is missing or empty.

- **Source Field:** The field to check.
- **Default Value:** Value to assign if blank.

Example: If status is missing, set "Pending".

The screenshot shows the 'ROW TRANSFORMERS' configuration window. Under 'Row-level transformer components', there is a section for 'Add Default Value'. The 'Transformer Name' is set to 'Add Default Value'. The 'Source Field' is 'Status'. The 'Default Value' is 'Sucess' (note the typo). There is an '+ Add' button.

Map New Values

Replace certain field values with new ones.

- **Source Field:** Field to transform.
- **Target Field:** New field to hold the mapped value.
- **Original Value:** Values to detect and replace.

Example: Map "Y" to "Yes", "N" to "No".

The screenshot shows the 'ROW TRANSFORMERS' configuration window. Under 'Row-level transformer components', there is a section for 'Map to New Values'. The 'Transformer Name' is set to 'Map to New Values'. The 'Configuration' section shows the 'Source Field' as 'Address' and the 'Target Field' as 'Street'. Below the 'Target Field', there is a text input for 'The target field name for the new value' with 'Original Value' entered. There is an '+ Add' button.

Remove Quotes

Remove unwanted quote characters from field values.

- **Enabled:** Enable or disable the transformer.
- **Source Fields:** Fields to process.
- **Quote Characters:** Characters to remove (e.g., " or ').

Example: Convert "ABC" → ABC.

The screenshot shows the 'ROW TRANSFORMERS' configuration interface. The first transformer is 'Remove Quote(s)'. It is marked as 'Enabled'. Under 'SOURCE FIELDS', the field 'ISIN' is listed. Under 'QUOTE CHARACTERS', the character '1' is listed. The interface includes a sidebar with icons for various functions and a top bar with a warning icon and a plus sign.

Replace Characters

Replace characters based on pattern matching.

- **Source Field:** Field to modify.
- **Pattern:** Regex to identify characters.
- **Replacement:** Value to replace matched pattern.

Example: Replace all - with / in a date.

The screenshot shows the 'ROW TRANSFORMERS' configuration interface. The first transformer is 'Replace Characters'. It is marked as 'Enabled'. Under 'Source Field', the field 'Country' is listed. Under 'Pattern', the regex '\S' is listed. Under 'Replacement', the character '/' is listed. The interface includes a sidebar with icons for various functions and a top bar with a warning icon and a plus sign. There are 'Delete' and 'Add' buttons for the configuration items.

Format Scientific Numbers

Convert scientific notation (e.g., 1.2E+05) to regular format.

- **Source Fields:** Fields that may contain scientific notation.

Example: Convert 1.23E+03 → 1230.

The screenshot shows the 'ROW TRANSFORMERS' configuration interface. The first transformer is 'Format Scientific Numbers'. The 'Transformer Name' field is set to 'Format Scientific Numbers'. Under the 'Configuration' section, the 'SOURCE FIELDS' list contains 'Amount'.

Add Recon Metadata

Attach internal metadata for reconciliation tracking.

- **DB Column Name:** Name of the metadata field to be added.

Example: Add SourceSystem field with value CoreBanking.

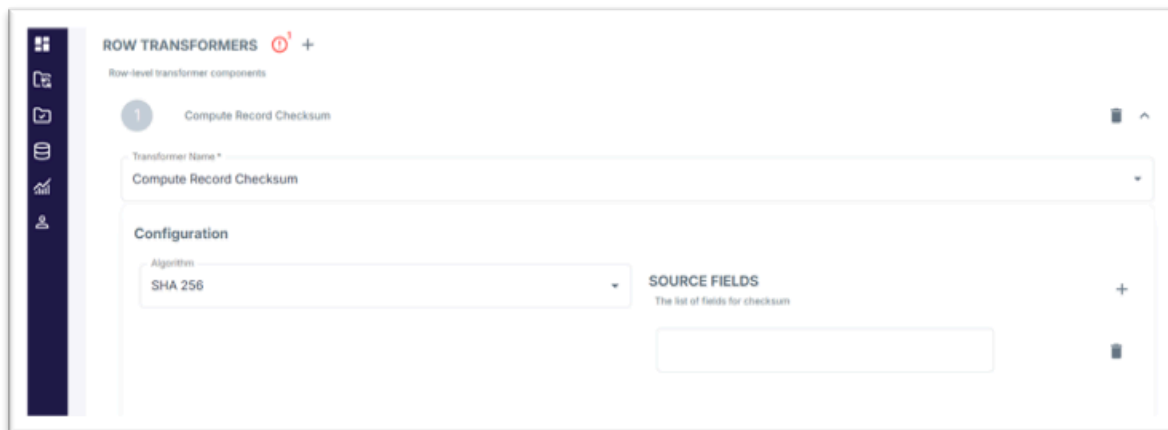
The screenshot shows the 'ROW TRANSFORMERS' configuration interface. The first transformer is 'Add Recon Metadata'. The 'Transformer Name' field is set to 'Add Recon Metadata'. Under the 'Configuration' section, the 'DB Column Name' is set to 'Column 1' and the 'Column 1' field is set to 'New address'. A red '+ Add' button is visible next to the 'Column 1' field.

Compute Records Checksum

Generate a unique hash for each row for verification.

- **Algorithm:** Algorithm used (e.g., SHA-256).
- **Source Fields:** Fields used to compute the hash.

Example: Hash of txn_id, amount, and date fields.



Compose New Fields

Create a new field by combining multiple fields with a pattern.

- **Target Field:** Name of the new field.
- **Output Pattern:** Format pattern using placeholders (%s).
- **Source Fields:** Fields to combine.

Example: Combine first_name and last_name → John Doe.



Scale Decimal Numbers

Multiply or divide numeric values to adjust precision.

- **Source Field:** Field to modify.
- **Scale By:** Value to multiply/divide the number.

Example: Multiply 1.23 by 100 to get 123.

The screenshot shows the 'ROW TRANSFORMERS' configuration page. Under 'Row-level transformer components', the first component is 'Scale Decimal Numbers'. The 'Transformer Name' is set to 'Scale Decimal Numbers'. In the 'Configuration' section, the 'Source Field' is 'Amount' and the 'Scale By' value is '100'. There is an '+ Add' button to the right of the 'Scale By' field.

Split Fields

Extract multiple values from a single field using regex.

- **Source Field:** The input field to split.
- **Named Regex Pattern:** Regex with named groups.

Example: Extract IFSC and Account from a field like IFSC:XXXX;ACC:1234.

The screenshot shows the 'ROW TRANSFORMERS' configuration page with the 'Transformation Details' tab selected. The first component is 'Split Fields'. The 'Transformer Name' is 'Split Fields'. In the 'Configuration' section, the 'Source Field' is 'Reference' and the 'Named Regex Pattern' field is empty. There is an '+ Add' button to the right of the 'Named Regex Pattern' field.

Mask Field Values

Anonymize sensitive data using templates.

- **Source Field:** Field to mask.
- **Template Name:** Predefined masking logic (select from dropdown).

Example: Mask PAN as ABCDE*F.

The screenshot shows the 'ROW TRANSFORMERS' configuration page. The first transformer is 'Mask Field Values'. The 'Transformer Name' field is set to 'Mask Field Values'. Under the 'Configuration' section, the 'Source Field' is 'Amount', the 'Template Name' is 'EMAIL', and the 'Mask Mode' is 'ANONYMIZE'. There are 'Delete' and 'Add' buttons.

Normalize XML

Converts nested XML structures into flat, readable rows.

- **Source Fields:** Field(s) containing XML content to flatten.

Example: Extract multiple <transaction> tags into separate rows.

The screenshot shows the 'ROW TRANSFORMERS' configuration page. The first transformer is 'Normalise XML'. The 'Transformer Name' field is set to 'Normalise XML'. Under the 'Configuration' section, the 'SOURCE FIELDS' list contains 'Amount'. There are 'Add' and 'Delete' buttons.

Construct XML

Builds an XML string using field values from a row.

- **Source Field:** Field to be converted into XML format.

Example: Convert fields like name, amount into <record> <name>John</name>
<amount>100</amount> </record>.

The screenshot shows the 'ROW TRANSFORMERS' configuration page. The first transformer is 'Construct XML'. The 'Transformer Name' field is set to 'Construct XML'. Under the 'Configuration' section, the 'Source Field' is 'Amount'. There is an 'Add' button.

Column Transformers

Applies transformations to data within a column based on its format (XML or JSON).

- **Source Field:** The field containing the structured content.
- **Field Type:** Select the format of the field: XML or JSON.

Example: For an XML column, extract a nested tag value like <status> or flatten the entire structure.



Lookup Transformation

Enriches a transaction record by resolving a field value from a pre-loaded master or reference data table. Use this processor when a target field cannot be derived from the incoming record alone — the value must be looked up from an external reference source such as a Bond Master, Holiday Calendar, or FX Rate table.

Reference data is loaded once at pipeline start and held in memory for the duration of the step, making this transformer suitable for high-volume workloads without per-row database round-trips.

Prerequisite

Before configuring a Lookup Transformation, a **Lookup Source** must exist under DataX > Admin > Lookup Sources for the reference table you want to query. See the Lookup Sources [Q](#) section.

Configuration

Each entry in the Lookup Transformation defines one target field to populate. Click + **Target Field** to add an entry. Each entry appears in the left panel identified by its target field name. Select an entry from the panel to configure its fields on the right. To remove an entry, click the delete icon next to its name in the panel.

Field	Required	Description
Lookup Source	Yes	The registered Lookup Source to query. Select from the drop-down of available sources.
Target Field	Yes	The field on the output record that will receive the resolved value.

Field	Required	Description
Value Column	Yes	The column name in the lookup table's result set that holds the value to copy into the target field.
Key Mappings	Yes (≥ 1)	One or more bindings that match the incoming record to a row in the lookup table. Each mapping specifies a Table Column (column in the lookup result set) and a Source Field (field on the transaction record). Multiple mappings form a composite key — all column values are concatenated in order to identify the matching row.
No Match Policy	No	What happens when no matching row is found in the lookup table. Fail Pipeline — stops the pipeline with an error. Do Nothing — forwards the record as is. Use Default Value — writes the configured default into the target field. Defaults to Fail Pipeline .
Default Value	Conditional	Required when No Match Policy is Use Default Value . The literal value written to the target field when no match is found.
Multi Match Policy	No	What happens when the composite key matches more than one row. First Match — uses the first row found. Last Match — uses the last row found. Maximum Value / Minimum Value — numeric aggregate over the value column. Fail on Multiple — applies the No Match Policy. Defaults to Fail on Multiple .

NOTE

If there is only one lookup mapping and **Source Field** value is null on an incoming record, the lookup is skipped for that record and the **No Match Policy** is applied directly.

Examples*Example 1. Bond Type from ISIN*

A pipeline processes trade records and needs to classify each trade by bond type.

- **Lookup Source:** Bond Master
- **Target Field:** BOND_TYPE
- **Value Column:** BOND_TYPE
- **Key Mappings:**
 - Table Column: ISIN → Source Field: ISIN
- **No Match Policy:** Use Default Value → Default Value: UNKNOWN
- **Multi Match Policy:** First Match

Example 2. Holiday Flag from Settlement Date and Country

A pipeline determines whether a settlement date falls on a public holiday in the settlement country.

- **Lookup Source:** Holiday Calendar
- **Target Field:** IS_HOLIDAY
- **Value Column:** IS_HOLIDAY
- **Key Mappings:**
 - Table Column: COUNTRY_CODE → Source Field: SETTLEMENT_COUNTRY
 - Table Column: HOLIDAY_DATE → Source Field: SETTLEMENT_DATE
- **No Match Policy:** Use Default Value → Default Value: false
- **Multi Match Policy:** First Match

Example 3. FX Rate from Currency Pair and Value Date

A pipeline enriches transactions with the reference FX rate applicable on the deal date.

- **Lookup Source:** FX Rate Master
- **Target Field:** FX_RATE
- **Value Column:** RATE
- **Key Mappings:**
 - Table Column: CCY_PAIR → Source Field: CURRENCY_PAIR
 - Table Column: RATE_DATE → Source Field: VALUE_DATE
- **No Match Policy:** Fail Pipeline
- **Multi Match Policy:** Fail on Multiple



Lookup Transformation

Writer Details

JDBC Writer

Used to write data into relational databases via JDBC.

- **Insert SQL Query:** Provide a valid SQL INSERT statement using placeholders (?) for each value to be written.



2.3 Edit & Delete Data Format

To edit or delete a Data Format, go to the respective Data Format and click on the corresponding icon (edit or delete).

image::Delete Dataformat, width=600]

Data Sets

Purpose

A Data Set is the logical container for ingested data.

Key Configuration

- Data Set ID
- Name/Description
- Linked Data Formats (at least one)

Common Guardrail

If a Data Set is already used by an Ops Process, deletion may be blocked until it is removed from the dependent process.

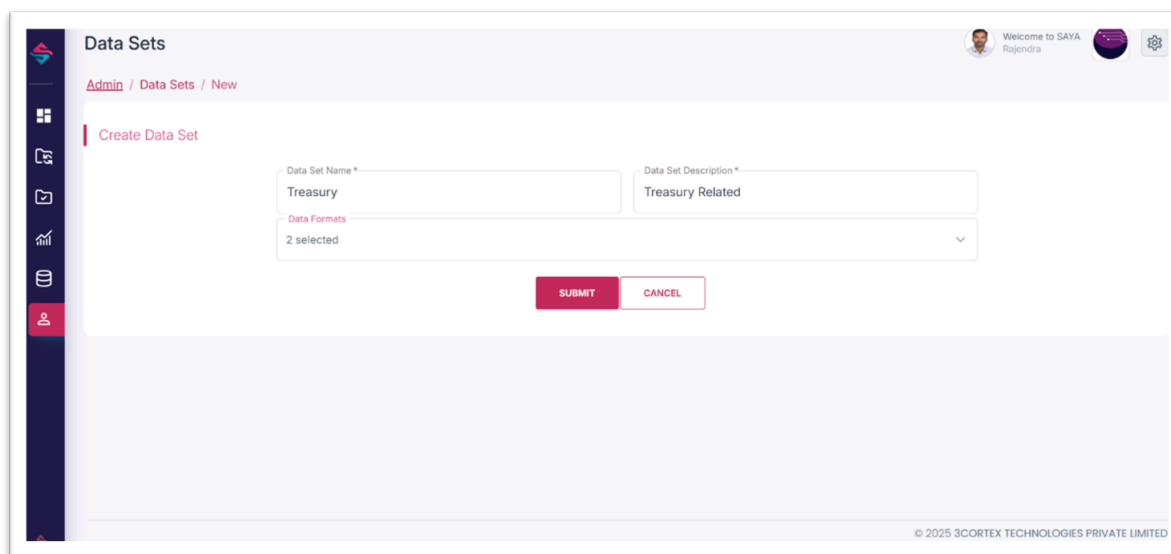
Dashboard→Admin→Click on Data Sets

Clicking on the Data Set tile on the Admin page allows users to create new data sets and edit or configure existing ones. This module allows users to configure various data formats to single dataset.

3.1 Create Data Set

1. Click on "Create New Dataset".
2. Enter a unique name for the dataset and a short description.

3. Select the required data formats from the dropdown list.
4. Click on "Submit" to save the dataset.



The screenshot shows the 'Data Sets' section of the SAYA platform. The breadcrumb trail is 'Admin / Data Sets / New'. The main heading is 'Create Data Set'. There are two input fields: 'Data Set Name *' with the value 'Treasury' and 'Data Set Description *' with the value 'Treasury Related'. Below these is a 'Data Formats' dropdown menu showing '2 selected'. At the bottom of the form are 'SUBMIT' and 'CANCEL' buttons. The footer of the page reads '© 2025 3CORTEX TECHNOLOGIES PRIVATE LIMITED'.

3.2 Edit & Delete Data Set

To edit or delete a Data Set, go to the respective Data Set and click on the corresponding icon (edit or delete).

image::Delete Dataset, width=600]



Copyright 2019 - 2026 3Cortex Technologies. All Rights Reserved.
[Privacy](#).

